

Table of contents

Knowledge Check Questions.....	4
Lab Tasks Completed.....	6
Conclusion.....	13

List of figures

Figure 1: C program with integer input/output.....	4
Figure 2: Output of C program.....	4
Figure 3: Linux command to create .c file and list files inside a folder.....	5
Figure 4: Linux command to write and view contents inside .c file. Also C language program to print Hello, World! and return 0.....	5
Figure 5: Linux command for 4 compilation stages (i.e Preprocessing, Compiling, Assembling and Linking and also command to disassemble the machine code to human readable code).....	6
Figure 6: Linux command to view permissions of a file.....	6
Figure 7: Linux command to view the output and return value.....	6
Figure 8: C language modified program to return 1 and print Hello!.....	7
Figure 9: Linux command to view the output and return value that was updated.....	7
Figure 10: C language program to accept user input and display it.....	7
Figure 11: Linux command to view the output and return value.....	8
Figure 12: Linux command of first compilation stage (preprocessing) and command to list that output file from that compilation.....	8
Figure 13: Linux command of second compilation stage (compiling) and command to list that output file from that compilation.....	8
Figure 14: Linux command of third compilation stage (assembling) and command to list that output file from that compilation.....	8
Figure 15: Linux command to disassemble the machine code into human readable code...	10
Figure 16: Linux command of fourth compilation stage(linking) and command to list that output file from that compilation.....	9
Figure 17: Linux command to view the permissions of hello file (final executable file).....	9
Figure 18: Linux command of three compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.....	10
Figure 19: Linux command of fourth compilation stage(linking) and command to view the hello1 file permission.....	11
Figure 20: C language program that returns integer value 50.....	11
Figure 21: Linux command of four compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.....	12
Figure 22: Linux command of four compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.....	12
Figure 23: Linux command to view the output and display the return value.....	12

Knowledge Check Questions

Q1. What is the purpose of the `#include` directive in C programming? Provide an example of its usage.

Ans: The purpose of `#include` directive in C programming is it tells the compiler to bring in code from a library (like `stdio.h`) so you can use it's functions such as `printf` and `scanf`.

Example:

```
#include <stdio.h>
```

Q2. Explain the difference between the four stages of compilation. What file extension is produced at each stage?

Ans: The four stages of compilation with it's produced file extension and purpose:

- Preprocessing (`.i`) → expands headers/macros.
 - Compilation (`.s`) → generates assembly code.
 - Assembling (`.o`) → converts assembly into machine code.
 - Linking (executable) → combines object files into a runnable program.
-

Q3. What does a return value of 0 from the `main()` function indicate to the operating system? What about a non-zero return value?

Ans: The return value of 0 and non-zero(1) from the `main()` function indicates to the operating system is:

- `return 0;` → success.
 - Non-zero (like `return 1;`) → error/failure.
-

Q4. Write a C program that:

- Declares an integer variable.
- Asks the user to input a value.
- Displays the entered value back to the user.

Ans:

```
#include <stdio.h>
int main() {
    int number;
    printf("Enter a number: ");
    scanf("%i", &number);
    printf("You entered: %i\n", number);
    return 0;
}
```

Figure 1: C program with integer input/output.

```
Enter a number: 5
You entered: 5
```

Figure 2: Output of C program.

Note: We can put %i or %d. The similarity is that both print an integer in decimal(base 10), while the difference is %i can print an integer in decimal(base 10), octal(base 8) and hexadecimal(base 16), but %d can only print an integer in decimal(base 10).

Q5. What GCC command would you use to:

- 1) Only preprocess a file without compiling?
- 2) Generate assembly code but not assemble it?
- 3) Create an object file without linking?

Ans: The GCC command I would use:

1. Preprocess only is → `gcc -E hello.c -o hello.i`
2. Generate assembly only is → `gcc -S hello.i -o hello.s`
3. Create object file only is → `gcc -c hello.s -o hello.o`

Q6. Explain why C is called a "compiled language" and how this differs from interpreted languages.

Ans: C is a compiled language because source code (any code/program we write and save in a “.c” file) is transformed into machine code before running.

The Interpreted languages differ from the C language because Interpreted languages (like Python) run line-by-line without creating machine code first.

Lab Tasks Completed

1) **Task 1:** Compiled hello.c with return 0 and verified exit status using echo \$?

Ans:

```
root@DESKTOP-OAP80K0:/batch38# touch hello.c
root@DESKTOP-OAP80K0:/batch38# ls
hello.c
```

Figure 3: Linux command to create .c file and list files inside a folder.

```
root@DESKTOP-OAP80K0:/batch38# nano hello.c
root@DESKTOP-OAP80K0:/batch38# cat hello.c
#include <stdio.h>

int main() {
    printf("Hello, World!\n");
    return 0;
}
```

Figure 4: Linux command to write and view contents inside .c file. Also C language program to print Hello, World! and return 0.

```

root@DESKTOP-OAP80K0:/batch38# gcc -E hello.c -o hello.i
root@DESKTOP-OAP80K0:/batch38# ls
root@DESKTOP-OAP80K0:/batch38# gcc -S hello.i -o hello.s
root@DESKTOP-OAP80K0:/batch38# ls
hello.c hello.i hello.s
root@DESKTOP-OAP80K0:/batch38# gcc -c hello.s -o hello.o
root@DESKTOP-OAP80K0:/batch38# ls
hello.c hello.i hello.o hello.s
root@DESKTOP-OAP80K0:/batch38# objdump -d hello.o

hello.o:      file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  e8 00 00 00 00      call   17 <main+0x17>
17:  b8 00 00 00 00      mov     $0x0,%eax
1c:  5d                   pop     %rbp
1d:  c3                   ret

root@DESKTOP-OAP80K0:/batch38# gcc hello.o -o hello

```

Figure 5: Linux command for 4 compilation stages (i.e Preprocessing, Compiling, Assembling and Linking and also command to disassemble the machine code to human readable code).

```

total 60
drwxr-xr-x  2 root root 4096 Sep 16 09:19 .
drwxr-xr-x 20 root root 4096 Sep 16 08:31 ..
-rwxr-xr-x  1 root root 15952 Sep 16 09:19 hello
-rw-r--r--  1 root root    75 Sep 16 08:33 hello.c
-rw-r--r--  1 root root 21429 Sep 16 08:47 hello.i
-rw-r--r--  1 root root 1496 Sep 16 09:02 hello.o

```

Figure 6: Linux command to view permissions of a file.

```

root@DESKTOP-OAP80K0:/batch38# ./hello
Hello, World!
root@DESKTOP-OAP80K0:/batch38# echo $?
0

```

Figure 7: Linux command to view the output and return value.

2) **Task 2:** Modified program to return 1 and observed different exit status.

Ans:

```
root@DESKTOP-OAP80K0:/batch38# nano hello.c
root@DESKTOP-OAP80K0:/batch38# cat hello.c
#include <stdio.h>

int main() {
    printf("Hello!\n");
    return 1;
}
```

Figure 8: C language modified program to return 1 and print Hello!.

```
root@DESKTOP-OAP80K0:/batch38# ./hello
Hello!
root@DESKTOP-OAP80K0:/batch38# echo $?
1
```

Figure 9: Linux command to view the output and return value that was updated..

3) **Task 3:** Created program accepting user input and displaying it.

Ans:

```
root@DESKTOP-OAP80K0:/batch38# touch hello1.c
root@DESKTOP-OAP80K0:/batch38# ls
hello hello.c hello.i hello.o hello.s hello1.c
root@DESKTOP-OAP80K0:/batch38# nano hello1.c
root@DESKTOP-OAP80K0:/batch38# cat hello1.c
#include <stdio.h>
int main() {
    int number;
    printf("Enter a number: ");
    scanf("%i", &number);
    printf("You entered: %i\n", number);
    return 0;
}
```

Figure 10: C language program to accept user input and display it.

```
root@DESKTOP-OAP80K0:/batch38# ./hello1
Enter a number: 5
You entered: 5
```

Figure 11: Linux command to view the output and return value.

4) **Task 4:** Executed all four compilation stages separately and documented output files.

Ans: Compilation stages of task 1:

```
root@DESKTOP-OAP80K0:/batch38# gcc -E hello.c -o hello.i
root@DESKTOP-OAP80K0:/batch38# ls
hello.c  hello.i
```

Figure 12: Linux command of first compilation stage (preprocessing) and command to list that output file from that compilation.

```
root@DESKTOP-OAP80K0:/batch38# gcc -S hello.i -o hello.s
root@DESKTOP-OAP80K0:/batch38# ls
hello.c  hello.i  hello.s
```

Figure 13: Linux command of second compilation stage (compiling) and command to list that output file from that compilation.

```
root@DESKTOP-OAP80K0:/batch38# gcc -c hello.s -o hello.o
root@DESKTOP-OAP80K0:/batch38# ls
hello.c  hello.i  hello.o  hello.s
```

Figure 14: Linux command of third compilation stage (assembling) and command to list that output file from that compilation.

```

root@DESKTOP-OAP80K0:/batch38# objdump -d hello.o

hello.o:          file format elf64-x86-64

Disassembly of section .text:

0000000000000000 <main>:
 0:  f3 0f 1e fa          endbr64
 4:  55                   push    %rbp
 5:  48 89 e5             mov     %rsp,%rbp
 8:  48 8d 05 00 00 00 00 lea     0x0(%rip),%rax      # f <main+0xf>
 f:  48 89 c7             mov     %rax,%rdi
12:  e8 00 00 00 00      call   17 <main+0x17>
17:  b8 00 00 00 00      mov     $0x0,%eax
1c:  5d                   pop     %rbp
1d:  c3                   ret

```

Figure 15: Linux command to disassemble the machine code into human readable code.

```

root@DESKTOP-OAP80K0:/batch38# gcc hello.o -o hello
root@DESKTOP-OAP80K0:/batch38# ls
hello  hello.c  hello.i  hello.o  hello.s

```

Figure 16: Linux command of fourth compilation stage(linking) and command to list that output file from that compilation.

```

total 60
drwxr-xr-x  2 root root  4096 Sep 16 09:19 .
drwxr-xr-x 20 root root  4096 Sep 16 08:31 ..
-rwxr-xr-x  1 root root 15952 Sep 16 09:19 hello
-rw-r--r--  1 root root   75 Sep 16 08:33 hello.c
-rw-r--r--  1 root root 21429 Sep 16 08:47 hello.i
-rw-r--r--  1 root root  1496 Sep 16 09:02 hello.o
-rw-r--r--  1 root root   661 Sep 16 08:58 hello.s

```

Figure 17: Linux command to view the permissions of hello file (final executable file).

Compilation stages of task 3:

```
root@DESKTOP-OAP80K0:/batch38# gcc -E hello1.c -o hello1.i
root@DESKTOP-OAP80K0:/batch38# ls
hello.c hello.i hello.o hello.s hello1.c hello1.i
root@DESKTOP-OAP80K0:/batch38# gcc -S hello1.i -o hello1.s
root@DESKTOP-OAP80K0:/batch38# ls
hello.c hello.i hello.o hello.s hello1.c hello1.i hello1.s
root@DESKTOP-OAP80K0:/batch38# gcc -c hello1.s -o hello1.o
root@DESKTOP-OAP80K0:/batch38# ls
hello.c hello.i hello.s hello1.i hello1.s
hello.c hello.o hello1.c hello1.o
root@DESKTOP-OAP80K0:/batch38# cat hello1.o
Enter a number: %iYou entered: %i
GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0GNUzRx
~E0C
F0000000P_00000000x00000000 .symtab.strtab.shstrtab.rela.text.data.bss.rodata.comment.note.gnu.property.rela.eh_frame @~`
&010%90'B
Rj0e@
h0
(6 root@DESKTOP-OAP80K0:/batch38# objdump -d hello1.o
hello1.o: file format elf64-x86-64
Disassembly of section .text:
```

Figure 18: Linux command of three compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.

```
root@DESKTOP-OAP80K0:/batch38# gcc hello1.o -o hello1
root@DESKTOP-OAP80K0:/batch38# ls -la
total 112
drwxr-xr-x  2 root root  4096 Sep 16 14:56 .
drwxr-xr-x 20 root root  4096 Sep 16 08:31 ..
-rwxr-xr-x  1 root root 15952 Sep 16 14:21 hello
-rw-r--r--  1 root root    68 Sep 16 14:18 hello.c
-rw-r--r--  1 root root 21422 Sep 16 14:19 hello.i
-rw-r--r--  1 root root  1488 Sep 16 14:20 hello.o
-rw-r--r--  1 root root  1116 Sep 16 14:45 hello.s
-rwxr-xr-x  1 root root 16056 Sep 16 14:56 hello1
-rw-r--r--  1 root root   149 Sep 16 14:51 hello1.c
-rw-r--r--  1 root root 21507 Sep 16 14:53 hello1.i
-rw-r--r--  1 root root  1816 Sep 16 14:54 hello1.o
-rw-r--r--  1 root root  1116 Sep 16 14:53 hello1.s
```

Figure 19: Linux command of fourth compilation stage(linking) and command to view the hello1 file permission.

5) **Task 5:** Compiled and ran program returning integer value 50

Ans:

```
root@DESKTOP-OAP8OK0:/batch38# touch hello2.c
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c
hello.c  hello.o  hello1   hello1.i  hello1.s
root@DESKTOP-OAP8OK0:/batch38# nano hello2.c
root@DESKTOP-OAP8OK0:/batch38# cat hello2.c
#include <stdio.h>
int main() {
    return 50;
}
```

Figure 20: C language program that returns integer value 50.

```
root@DESKTOP-OAP8OK0:/batch38# gcc -E hello2.c -o hello2.i
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -S hello2.i -o hello2.s
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.s
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i
root@DESKTOP-OAP8OK0:/batch38# gcc -c hello2.s -o hello2.o
root@DESKTOP-OAP8OK0:/batch38# ls
hello    hello.i  hello.s  hello1.c  hello1.o  hello2.c  hello2.o
hello.c  hello.o  hello1   hello1.i  hello1.s  hello2.i  hello2.s
root@DESKTOP-OAP8OK0:/batch38# cat hello2.o
ELF>@@@

    UH2]GCC: (Ubuntu 15.2.0-16ubuntu1) 15.2.0GNUzRx
F                                     EC
hello2.cmain .symtab.strtab.shstrtab.text.data.bss.comment.note.GNU-stack.note.gnu.property.rela.eh_frame!00,00'5vEx]X@@
0Xgroot@DESKTOP-OAP8OK0:/batch38# objdump -d hello2.o
```

Figure 21: Linux command of four compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.

```

root@DESKTOP-OAP80K0:/batch38# gcc hello2.o -o hello2
root@DESKTOP-OAP80K0:/batch38# ls -la
total 164
drwxr-xr-x  2 root root  4096 Sep 16 15:12 .
drwxr-xr-x 20 root root  4096 Sep 16 08:31 ..
-rwxr-xr-x  1 root root 15952 Sep 16 14:21 hello
-rw-r--r--  1 root root    68 Sep 16 14:18 hello.c
-rw-r--r--  1 root root 21422 Sep 16 14:19 hello.i
-rw-r--r--  1 root root  1488 Sep 16 14:20 hello.o
-rw-r--r--  1 root root  1116 Sep 16 14:45 hello.s
-rwxr-xr-x  1 root root 16056 Sep 16 14:56 hello1
-rw-r--r--  1 root root   149 Sep 16 14:51 hello1.c
-rw-r--r--  1 root root 21507 Sep 16 14:53 hello1.i
-rw-r--r--  1 root root  1816 Sep 16 14:54 hello1.o
-rw-r--r--  1 root root  1116 Sep 16 14:53 hello1.s
-rwxr-xr-x  1 root root 15768 Sep 16 15:12 hello2

```

Figure 22: Linux command of four compilation stages (preprocessing, compiling, assembling) and command to disassemble the machine code into human readable code.

```

root@DESKTOP-OAP80K0:/batch38# ./hello2
root@DESKTOP-OAP80K0:/batch38# echo $?
50

```

Figure 23: Linux command to view the output and display the return value.

Conclusion

This lab exercise provided hands-on experience with C programming fundamentals, including program structure, compilation process, and GCC usage. Understanding these concepts is essential for systems programming and operating system development. The four-stage compilation process demonstrates how high-level code is transformed into machine-executable instructions.